Q1

EAD UNCOMMITTED

- 说明: 最低级别的隔离级别，事务可以读取尚未提交的数据。
- 结果:
- 语句 2（会话 1）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=2000（更新后的值）
- 语句 4（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=2000（读取到会话 1 更新的值）
- 语句 5（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=2000

```sql
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SET AUTOCOMMIT = 0;
START TRANSACTION;
UPDATE account SET money = money + 1000 WHERE id = 1;
SELECT * FROM account WHERE id = 1;
```

count 1 | account 2

ELECT * FROM account WHERE id = 1 | Enter a SQL expression to filter results (use Ctrl+Space

id	name	money
1	tom	1,000

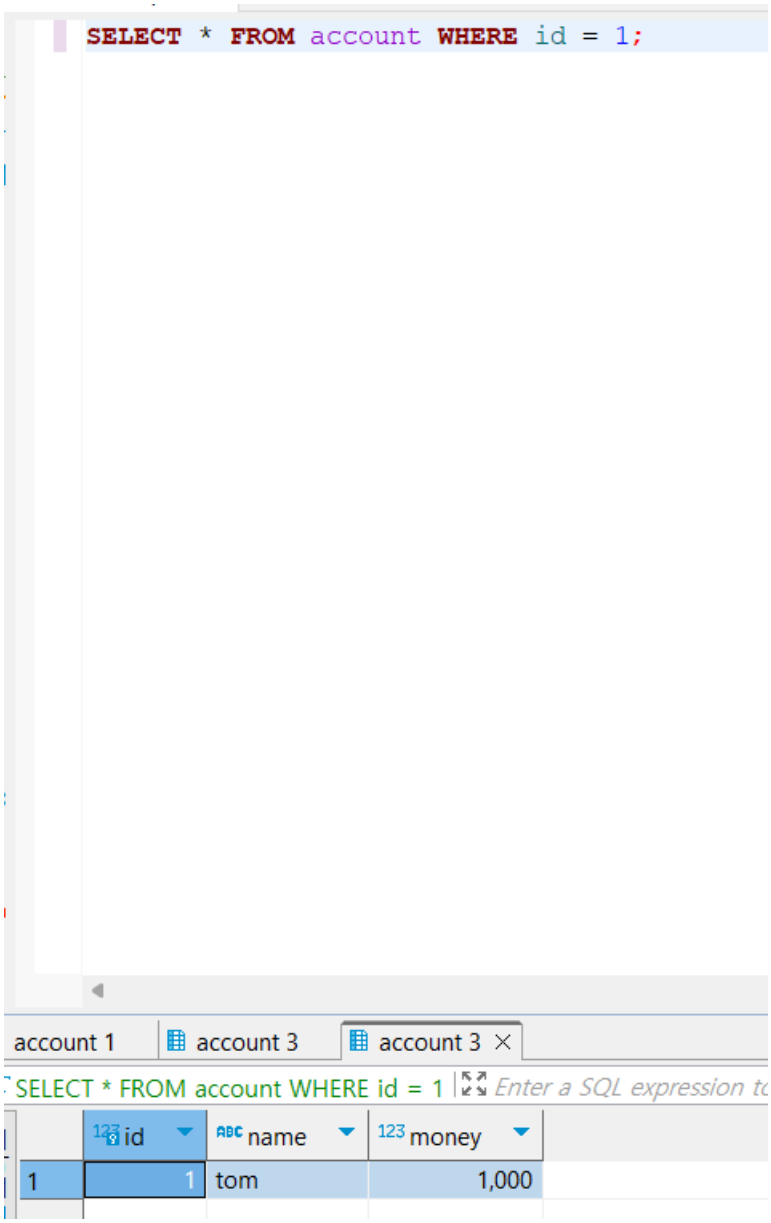
*<lab7> Script-8 | *<lab7> Script-9

```sql
set session transaction isolation level read uncommitted;
start transaction;
select * from account where id = 1;
```

account 1

select * from account wh | Enter a SQL expression to filter results (use Ctrl+S

	id	name	money
1	1	tom	2,000



Q2

READ COMMITTED

- 说明: 事务只能读取已提交的数据。
- 结果:
- 语句 2（会话 1）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=2000
- 语句 4（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=1000（因为会话 1 的事务尚未提交）

- 语句 5（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=1000

REPEATABLE READ

- 说明: 事务读取的都是事务开始时的数据，即使在事务内多次读取相同的数据，结果也相同。
- 结果:
- 语句 2（会话 1）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=2000
- 语句 4（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=1000（因为会话 1 的事务尚未提交）
- 语句 5（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=1000

SERIALIZABLE

- 说明: 最高级别的隔离级别，事务像串行执行一样运行。
- 结果:
- 语句 2（会话 1）：SELECT * FROM account WHERE id = 1;
- 结果：id=1，name='tom'，money=2000
- 语句 4（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：阻塞（等待会话 1 的事务完成）
- 语句 5（会话 2）：SELECT * FROM account WHERE id = 1;
- 结果：阻塞（等待会话 1 的事务完成）

*<lab7> Script-8 × *<lab7> Script-9

```sql
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SET AUTOCOMMIT = 0;
START TRANSACTION;
UPDATE account SET money = money + 1000 WHERE id = 1;
SELECT * FROM account WHERE id = 1;
```

Statistics 1 account 2 ×

SELECT * FROM accoun Enter a SQL expression to filter results (use Ctrl

	id	name	money
1	1	tom	1,000

*<lab7> Script-8 | *<lab7> Script-9

```
set session transaction isolation level read committed ;
start transaction;
select * from account where id = 1;
```

account 1

select * from account wh | Enter a SQL expression to filter results (use Ctrl+S

	id	name	money
1	1	tom	2,000

*<lab7> Script-8 | *<lab7> Script-9

```
SELECT * FROM account WHERE id = 1;
```

account 1 | Statistics 2 | account 3 | account 4 | acc

SELECT * FROM accoun | Enter a SQL expression to filter results (use

	id	name	money
1	1	tom	1,000

*<lab7> Script-8 × *<lab7> Script-9

```sql
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SET AUTOCOMMIT = 0;
START TRANSACTION;
UPDATE account SET money = money + 1000 WHERE id = 1;
SELECT * FROM account WHERE id = 1;
```

account 5 ×

SELECT * FROM accoun | Enter a SQL expression to filter results (use Ctrl-

그리드

	id	name	money
1	1	tom	1,000

*<lab7> Script-8 | *<lab7> Script-9

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT * FROM account WHERE id = 1;
```

account 1 | Statistics 2 | account 3 | account 4 | account 5 | account 6

SELECT * FROM accoun | Enter a SQL expression to filter results (use Ctrl

	id	name	money
1	1	tom	1,000

*<lab7> Script-8 | *<lab7> Script-9

```
SELECT * FROM account WHERE id = 1;
```

Statistics 2 | account 3 | account 4 | account 5 | account

SELECT * FROM accoun | Enter a SQL expression to filter results (use Ctrl-

결과

	id	name	money
1	1	tom	1,000

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SET AUTOCOMMIT = 0;
START TRANSACTION;
UPDATE account SET money = money + 1000 WHERE id = 1;
SELECT * FROM account WHERE id = 1;
```

Statistics 1 | Statistics 2 | account 3

SELECT * FROM accoun | Enter a SQL expression to filter results (use Ctrl-

	id	name	money
1	1	tom	1,000

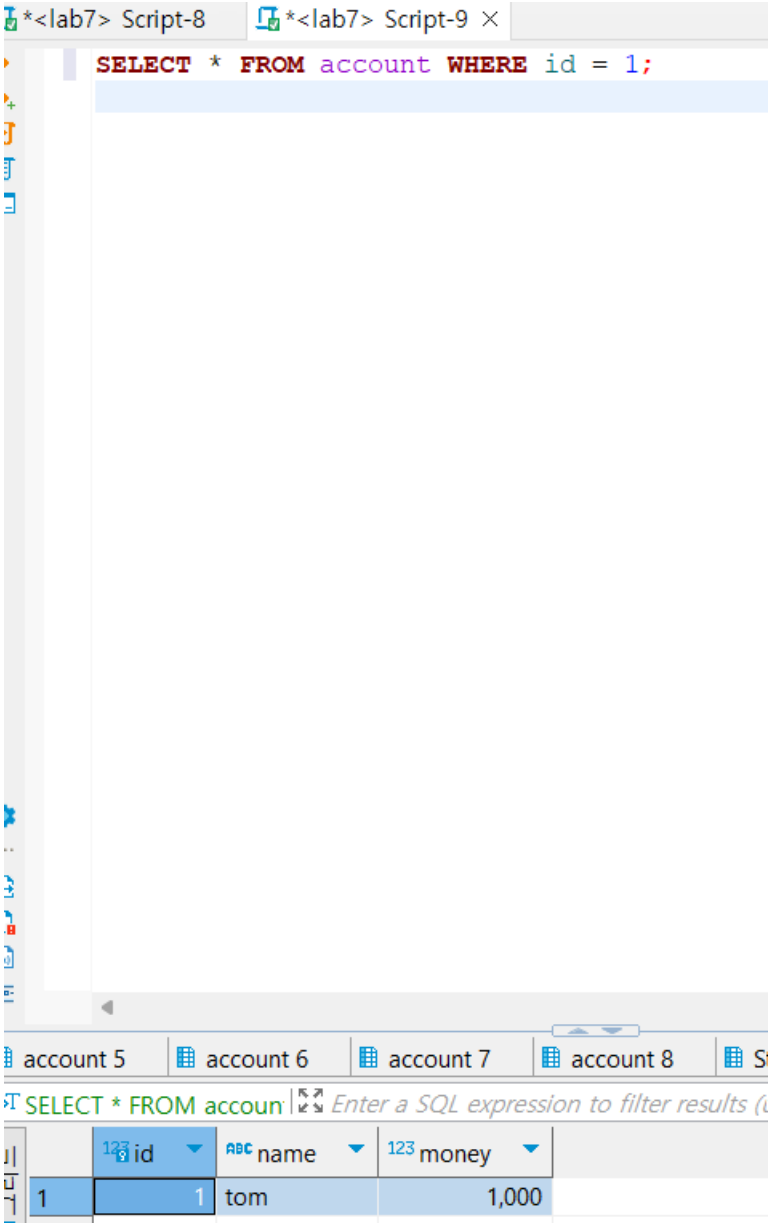
*<lab7> Script-8 *<lab7> Script-9

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
SELECT * FROM account WHERE id = 1;
```

account 3 | account 4 | account 5 | account 6 | account 7 | account

SELECT * FROM accoun | Enter a SQL expression to filter results (use Ctrl-

	id	name	money
1	1	tom	1,000



Q3

READ COMMITTED

第一次查询结果是 money = 0。（初始状态）第二次查询结果是 money = 1000。（会话 2 的更新提交后）

在这个隔离级别，事务只能读取已提交的数据。会话 1 最初读取的数据是事务开始前提交的数据。会话 2 提交更新后，会话 1 读取的是最新提交的数据。

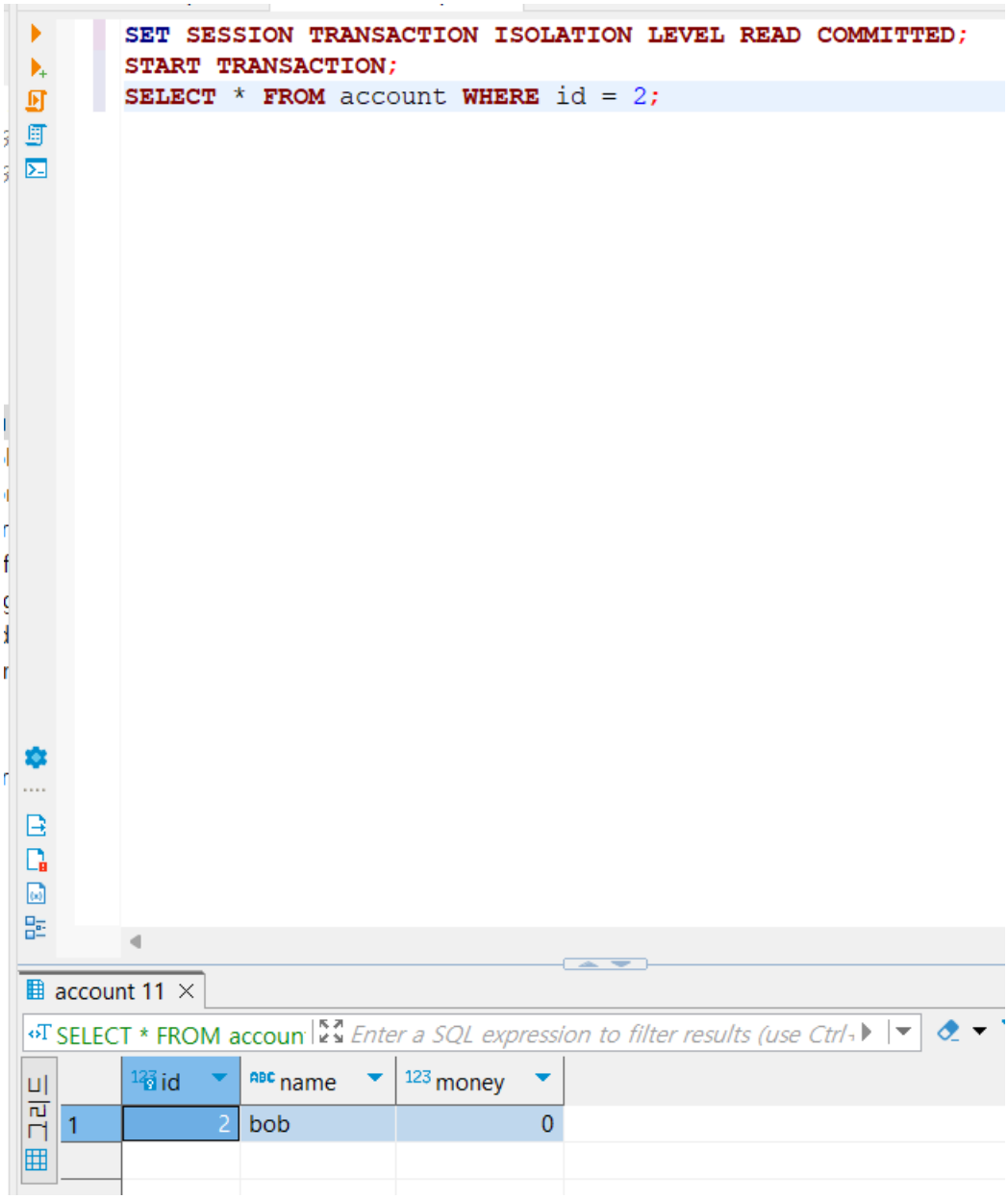
*<lab/> Script-8 *<lab/> Script-9

```
select * from account where id = 2
```

account 1

select * from account wh Enter a SQL expression to filter results (u

	id	name	money
1	2	bob	1,000



Q4

REPEATABLE READ

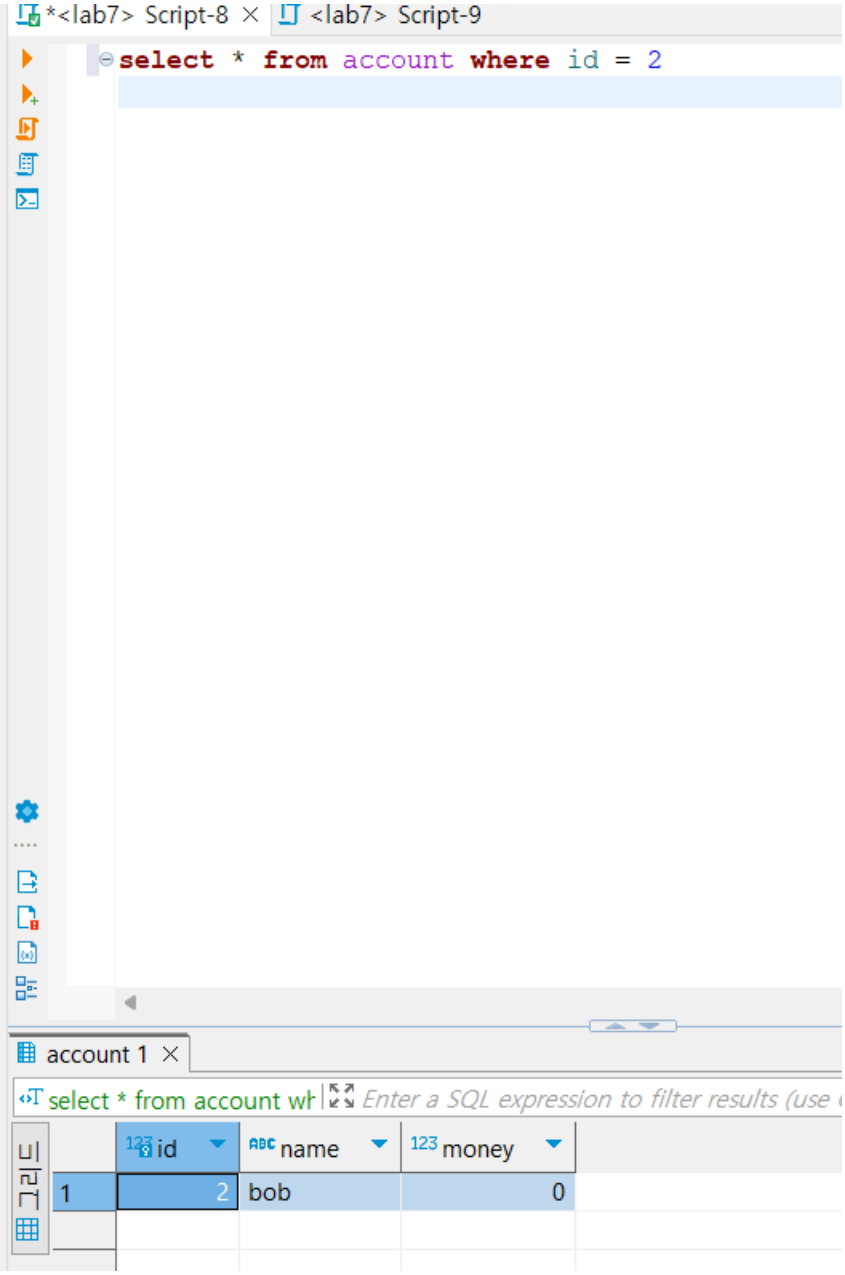
第一次查询结果是 money = 0。（初始状态）第二次查询结果也是 money = 0。（会话 1 的事务期间数据一致）

在这个隔离级别，事务开始后读取的所有数据都是一致的。会话 1 在事务期间多次查询，读取的都是相同的数据，因为事务开始时的数据快照被保持。

SERIALIZABLE

第一次查询结果是 money = 0。（初始状态）会话 2 执行时，UPDATE 语句被阻塞或等待。会话 1 的事务完成之前，会话 2 不能修改数据。

在这个隔离级别，提供最高级别的隔离，事务像串行执行一样运行。当另一个事务试图访问当前事务正在读取的数据时，会被阻塞或等待。



*<lab7> Script-8 × <lab7> Script-9

```
set session transaction isolation level repeatable read ;
start transaction;
select * from account where id = 2;
```

account 1 ×

select * from account wh | Enter a SQL expression to filter results (use Ctrl+S

	id	name	money
1	2	bob	0

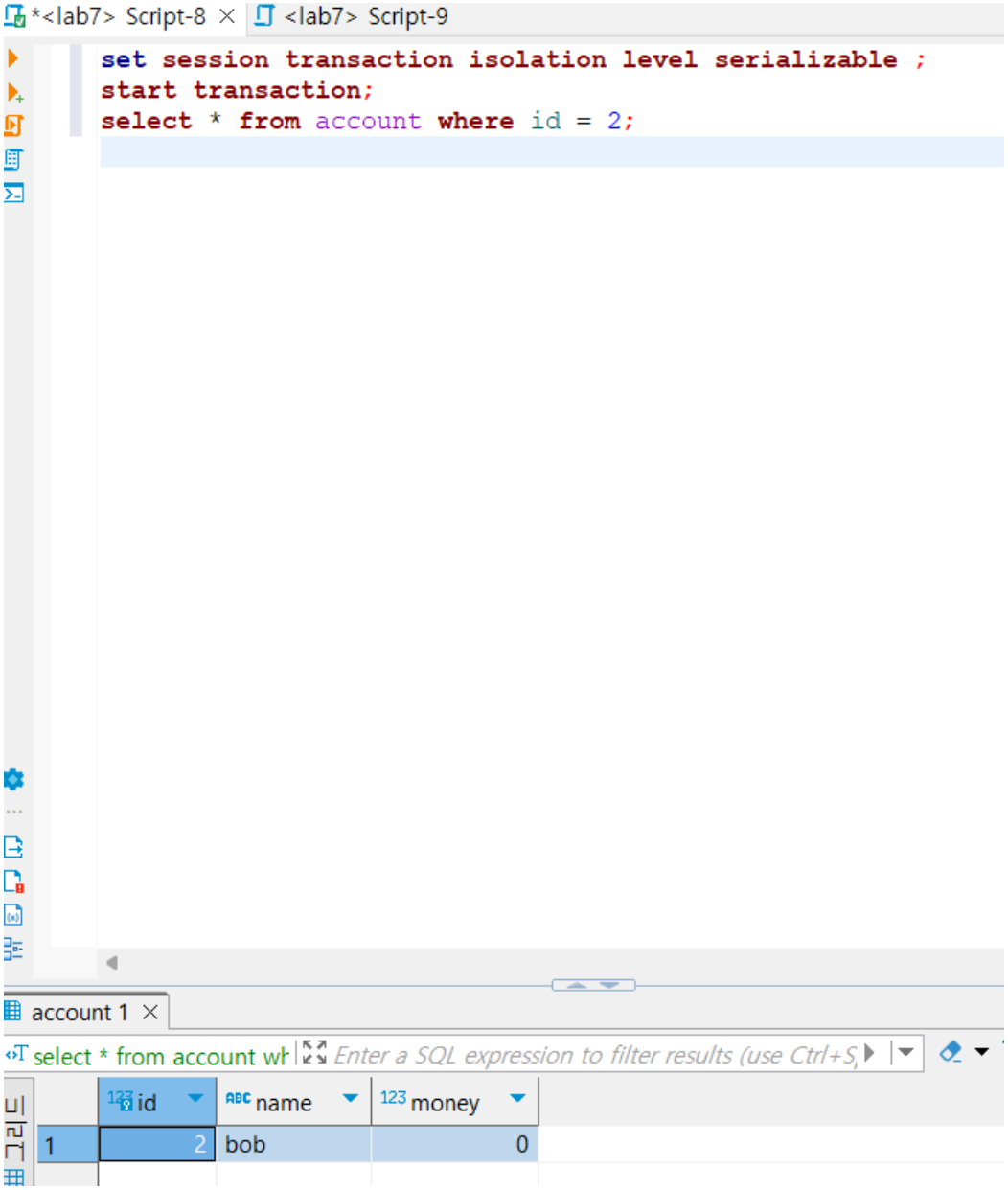
*<lab7> Script-8 × <lab7> Script-9

```
select * from account where id = 2
```

account 1 ×

select * from account wh | Enter a SQL expression to filter results (use

	id	name	money
1	2	bob	0



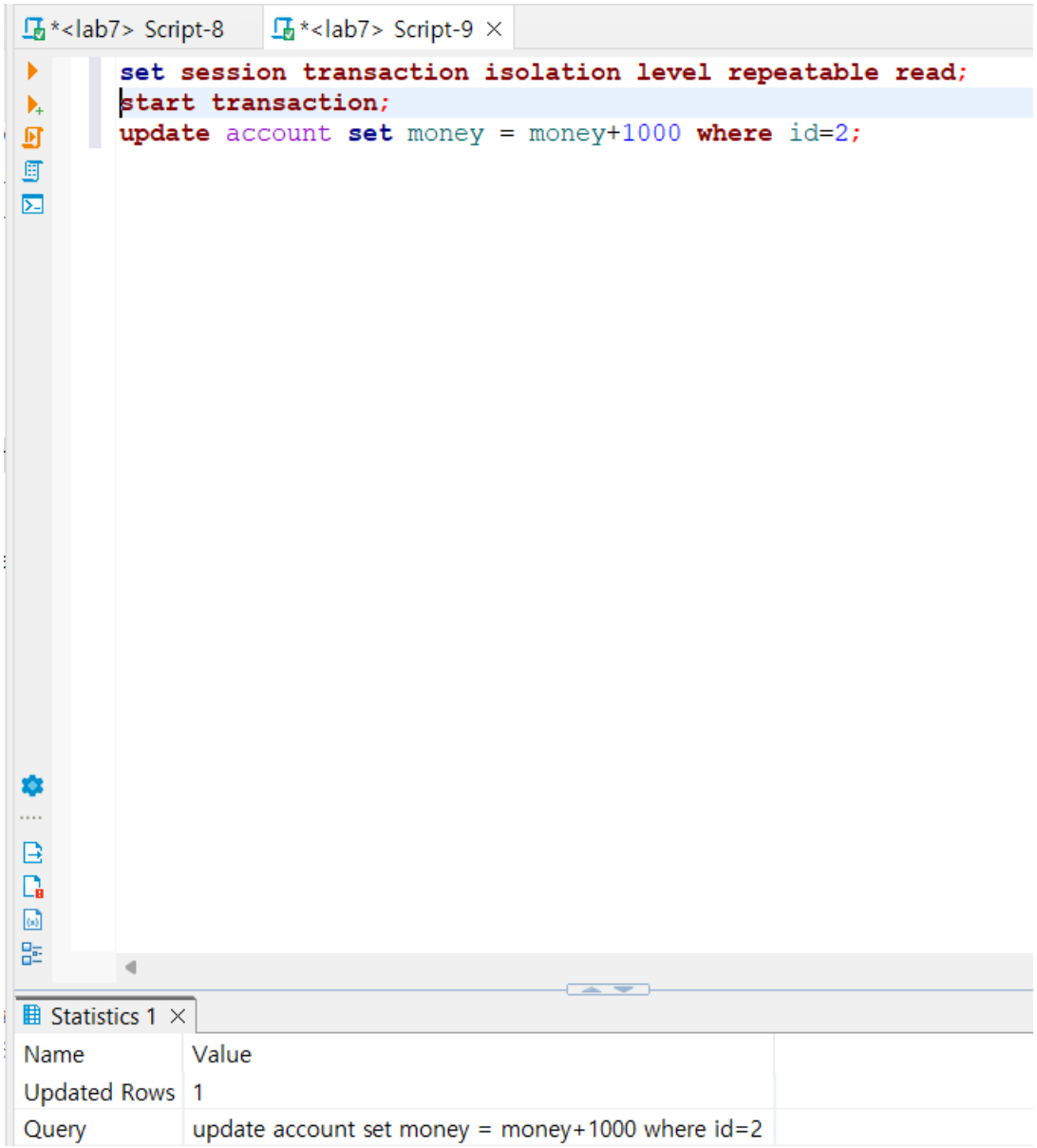
Q5

id=2 记录上会加共享锁（S 锁），其他事务（会话 2）不能修改该记录。

尝试执行 UPDATE account SET money = money + 1000 WHERE id = 2; : 阻塞

因为会话 1 在 id=2 记录上加了共享锁，所以会话 2 不能修改该记录，必须等待会话 1 提交或回滚事务。

会话 1 执行 COMMIT 后，共享锁解除。然后会话 2 阻塞的 UPDATE 命令执行，money 值更新为



1000。

Q6

在REPEATABLE READ 离级别下，事务开始时的数据会一直显示，所以即使其他事务插入了新数据，已开始的事务也看不到这些数据。

*<lab7> Script-8 × *<lab7> Script-9

```sql
select * from account where id < 10;
```

account 1 ×

select * from account wh | Enter a SQL expression to filter results (use Ctrl+

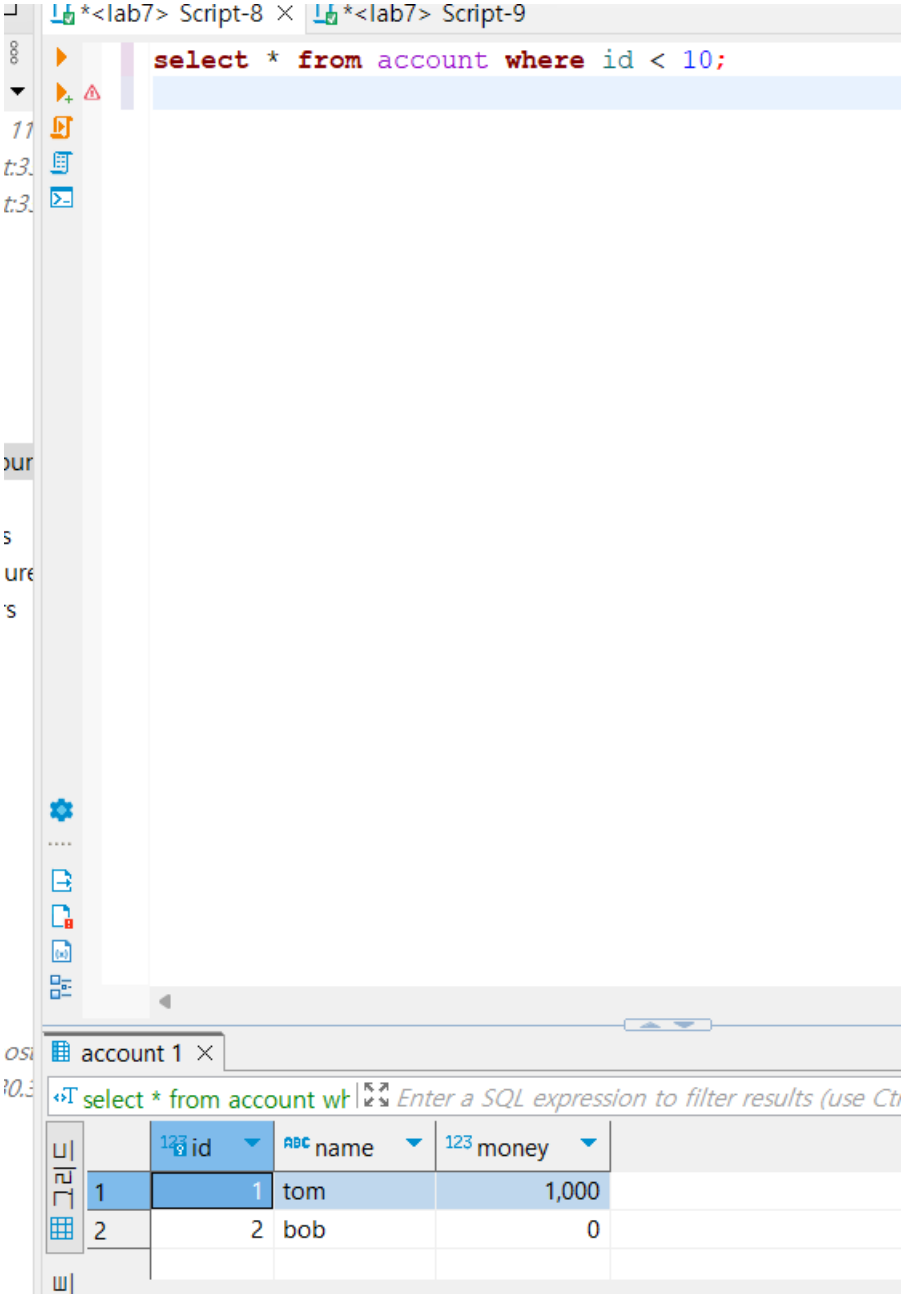
	id	name	money
1	1	tom	1,000
2	2	bob	0

```
set session transaction isolation level repeatable read; // 语句1
start transaction;
set autocommit = 0;
select * from account where id < 10;
```

account 1

select * from account wh | Enter a SQL expression to filter results (use Ctrl+S

	id	name	money
1	1	tom	1,000
2	2	bob	0



Q7

在SERIALIZABLE 离级别下，这是最严格的隔离级别，事务执行期间其他事务无法插入或修改数据。因此，新数据插入后，原始事务不会受到影响。

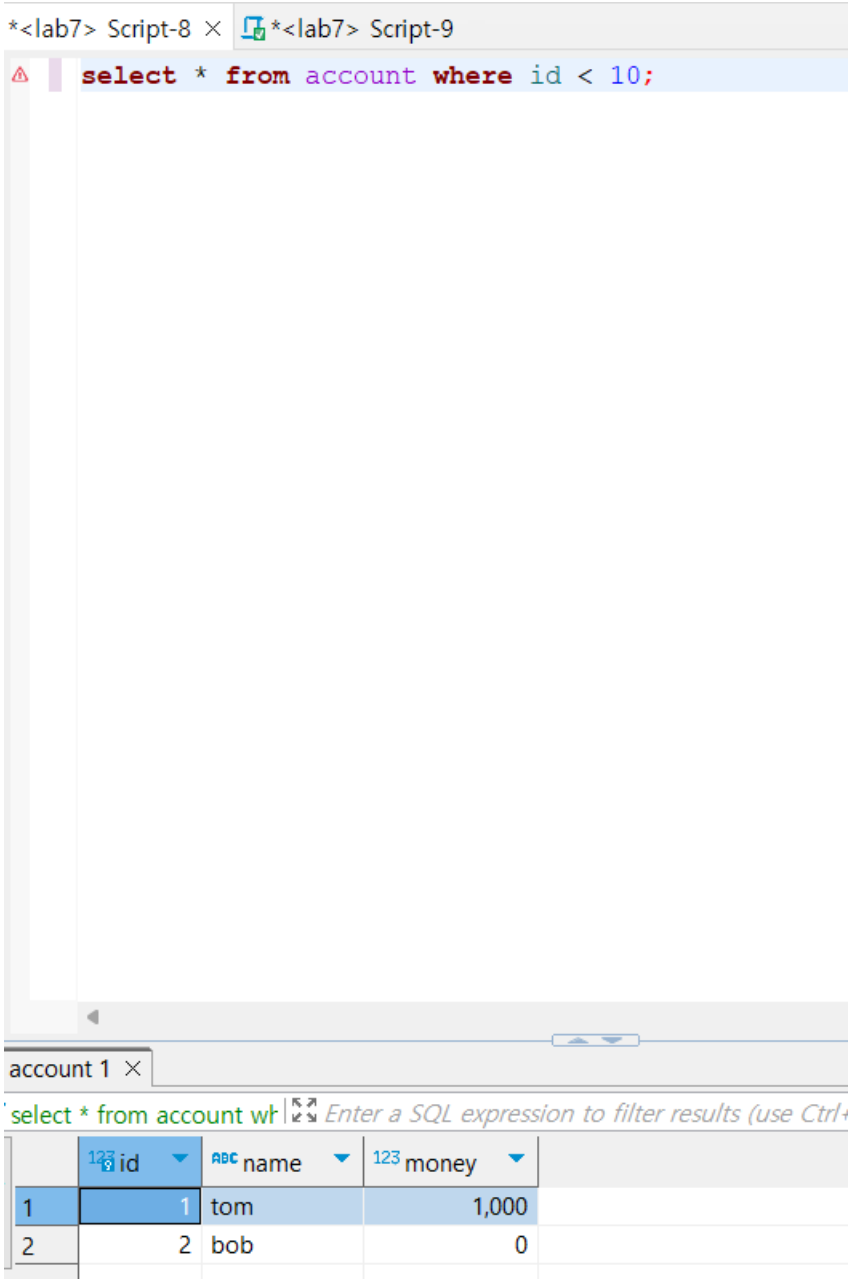
*<lab7> Script-8 × *<lab7> Script-9

```sql
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;
SET AUTOCOMMIT = 0;
SELECT * FROM account WHERE id < 10;
```

account 1 ×

select * from account wh | Enter a SQL expression to filter results (use Ctrl+S

	id	name	money
1	1	tom	1,000
2	2	bob	0



Q8

LOCK IN SHARE MODE 加共享锁，阻止其他事务修改数据，直到共享锁解除，其他写操作会被阻塞。

Q9

尝试在记录上加 X 锁，但会话 1 已经持有 X 锁，因此会被阻塞。这导致事务在完成前等待。会话 1 提交事务后，id=1 记录上的锁解除。

会话 1 提交后，会话 2 的语句 2 和 3 重新执行，成功删除 id=1 记录。

*<lab7> Script-8 | *<lab7> Script-9 × | account

```
set session transaction isolation level serializable;
start transaction;
delete from account where id=1;
```

Statistics 1 ×

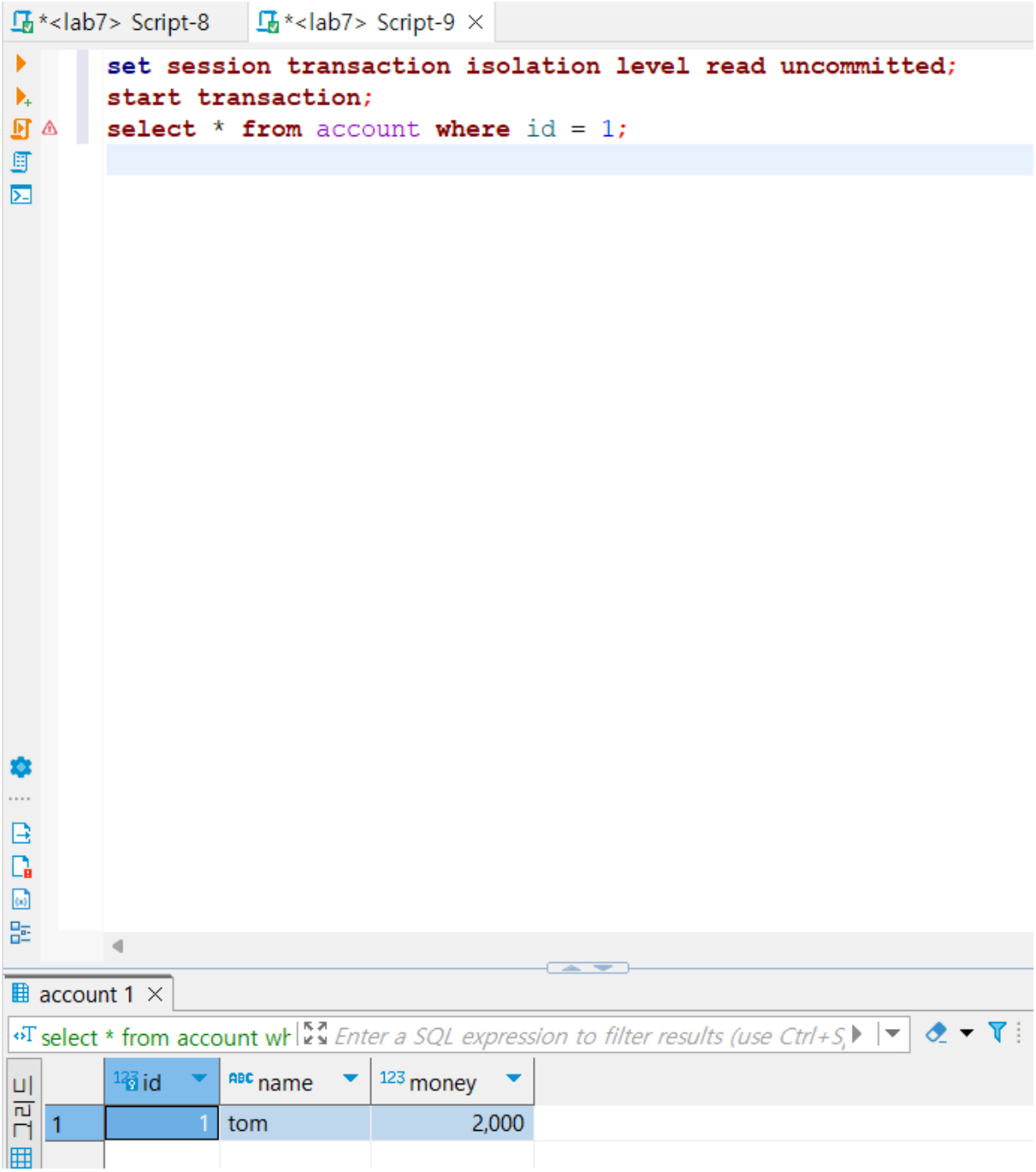
Name	Value
Updated Rows	0
Query	delete from account where id=1
Start time	Sun May 19 17:49:31 CST 2024
Finish time	Sun May 19 17:49:31 CST 2024

*<lab7> Script-8 | *<lab7> Script-9 | account

```
delete from account where id=1;
```

Statistics 1

Name	Value
Updated Rows	0
Query	delete from account where id=1
Start time	Sun May 19 17:50:48 CST 2024
Finish time	Sun May 19 17:50:48 CST 2024



Q10

死锁发生原因

会话 A 对 tableA 的 id=1 持有排他锁（X 锁）。

会话 B 对 tableB 的 id=1 持有排他锁（X 锁）。

会话 A 试图更新 tableB 的 id=1，但该记录已被会话 B 锁定。

同时，会话 B 试图更新 tableA 的 id=1，但该记录已被会话 A 锁定。

这导致两个事务互相等待对方的锁，从而发生死锁。